B2B Travel Booking & Revenue Analytics

PHASE 3 - FX & CURRENCY NORMALIZATION

Complete Beginner-Friendly Documentation

What to do | How to do it | Why to do it | Concepts | SQL | Validation | Analyst thinking

Core Phase 3 idea Convert every monetary event into one reporting currency at the date that matches that event, then validate the conversion before any company-wide financial aggregation.

Project stack: PostgreSQL / pgAdmin | SQL | Python | Power BI | Excel

Learning mode: beginner-friendly, step-by-step

1. Phase 3 Overview

Phase 3 solves the multi-currency problem. The clean booking, cancellation and payment tables are trustworthy structurally, but their monetary values are still expressed in many different currencies. Before total GBV, revenue, refunds or payments can be compared across the whole business, all monetary events must be normalized to one reporting currency.

What Phase 3 answers

- Why can we not add INR, USD, EUR, AED and other currency amounts directly?
- What is an FX rate and what should the grain of an FX table be?
- How do we generate monthly FX rows for every currency?
- How do we convert booking amounts using booking-month FX?
- How do we convert refunds using cancellation-month FX?
- How do we convert payments using payment-month FX?
- Why should payment transactions be converted before they are aggregated?
- How do we detect missing FX mappings and conversion mismatches?
- How do we reconcile payment summaries to valid and excluded bookings?
- How do we prepare normalized data for Phase 4 data marts?

Check	Verified result	Meaning
Clean booking rows	119,877	Analysis-ready bookings entering Phase 3
Base currencies	15	All booking currencies represented in the FX mapping
Initial monthly FX rows	360	24 booking months x 15 currencies
Extended monthly FX rows	435	29 months through May 2026 x 15 currencies
Booking FX missing rows	0	Every clean booking matched an FX rate
Cancellation FX missing rows	0	Every clean cancellation matched an FX rate
Payment transactions	127,189	All payment transactions converted at transaction level
Payment missing FX / currency	0 / 0	Every payment found both currency and FX rate
Booking-level payment summaries	120,000	Exactly one summary row per real booking
True orphan payment bookings	0	Every payment summary belongs to a real booking

Important The FX rates used in this portfolio project are synthetic learning rates, not official historical market rates. In a production system, replace them with governed historical FX data.

2. The Core Multi-Currency Concept

WHAT TO DO	WHY	HOW
Choose one reporting currency and normalize every financial amount before company-wide aggregation.	Amounts in different currencies do not share the same unit. Adding them directly creates a number with no valid financial meaning.	Use INR as the reporting currency and multiply each local amount by a rate_to_inr chosen from the month of the relevant financial event.

Why raw multi-currency sums are wrong

Suppose one booking is 10,000 INR and another is 1,000 USD. Writing 10,000 + 1,000 = 11,000 is mathematically possible but financially meaningless because the two values have different units.

Formula Amount in INR = Amount in local currency x FX rate to INR

Example: 100 USD x 83 INR per USD = 8,300 INR. For INR itself, the rate is always 1 because 1 INR = 1 INR.

Use the event date that matches the money

Financial event	Date used for FX	Reason
Booking GBV / supplier cost / discount / net revenue	Booking month	These values belong to the booking event
Refund amount / cancellation fee	Cancellation month	These values belong to the cancellation event
Payment transaction amount	Payment month	The cash movement occurs on the payment transaction date

Analyst rule Choose the FX date based on the economic event being converted, not simply the date that is easiest to join.

3. Lessons 1-4: Build the FX Foundation

Lesson 1: Create the analytics schema

WHAT TO DO	WHY	HOW
Create the analytics schema.	Separate business-ready analytical objects from the clean layer.	Use CREATE SCHEMA IF NOT EXISTS so setup can be rerun safely.

```
CREATE SCHEMA IF NOT EXISTS analytics;
```

Concept: analytics layer CLEAN stores trusted business records. ANALYTICS stores reporting calculations, FX-normalized facts and later data marts.

Lesson 2: Determine the booking date range

WHAT TO DO	WHY	HOW
Find the first and last clean booking dates.	The FX table must cover every booking month that will be converted.	Use MIN() and MAX() on booking_dt.

```
SELECT
    MIN(booking_dt) AS first_booking_date,
    MAX(booking_dt) AS last_booking_date
FROM clean.fact_bookings;
```

Check	Verified result	Meaning
First booking date	2024-01-01	FX coverage must begin in Jan 2024
Last booking date	2025-12-31	Booking FX coverage must include Dec 2025

Lesson 3: Create the base FX-rate table

WHAT TO DO
Store one synthetic base INR rate for each of the 15 currencies.

WHY
A controlled lookup table avoids scattering hard-coded conversion logic across many queries.

HOW
Use currency as a primary key and store base_rate_to_inr as NUMERIC.

```
CREATE TABLE analytics.fx_base_rates (
    currency VARCHAR(3) PRIMARY KEY,
    base_rate_to_inr NUMERIC(12,4)
);

INSERT INTO analytics.fx_base_rates (currency, base_rate_to_inr)
VALUES
    ('INR', 1.0000), ('USD', 83.0000), ('EUR', 90.0000),
    ('GBP', 105.0000), ('AED', 22.6000), ('SAR', 22.1000),
    ('QAR', 22.8000), ('AUD', 54.0000), ('SGD', 62.0000),
    ('MYR', 17.7000), ('JPY', 0.5600), ('THB', 2.3000),
    ('IDR', 0.0053), ('TRY', 2.6000), ('EGP', 1.7000);
```

Check	Verified result
Currency count	15

Primary-key lesson A unique currency key prevents duplicate base mappings such as two USD rows, which could later multiply booking records during a join.

Lesson 4: Expand base rates into monthly FX rates

WHAT TO DO
Create one FX row per currency per month.

WHY
Financial events need time-aware FX mappings, and later joins require a stable month + currency grain.

HOW
Generate the month list with GENERATE_SERIES() and CROSS JOIN it to all currencies.

```
CREATE TABLE analytics.fx_rates AS
WITH month_range AS (
    SELECT
        DATE_TRUNC('month', MIN(booking_dt))::date AS start_month,
        DATE_TRUNC('month', MAX(booking_dt))::date AS end_month
    FROM clean.fact_bookings
),
months AS (
    SELECT GENERATE_SERIES(
        start_month,
        end_month,
        INTERVAL '1 month'
    )::date AS fx_month
    FROM month_range
)
SELECT
    m.fx_month,
    f.currency,
    f.base_rate_to_inr AS rate_to_inr
FROM months AS m
CROSS JOIN analytics.fx_base_rates AS f;
```

Check	Verified result	Why
Initial FX rows	360	24 months x 15 currencies
Distinct months	24	Jan 2024 through Dec 2025
Distinct currencies	15	All source booking currencies covered

Concept: CROSS JOIN Every month is paired with every currency. 24 month rows x 15 currency rows = 360 combinations.

4. Lessons 5-7: Normalize Booking Financials

Lesson 5: Join FX rates to bookings

WHAT TO DO
Create analytics.fact_bookings_fx with INR versions of booking monetary columns.

WHY
Company-wide GBV and revenue calculations are only meaningful after all bookings share one currency.

HOW
Join on booking month + currency and preserve both local and INR values.

```
CREATE TABLE analytics.fact_bookings_fx AS
SELECT
    b.*,
    DATE_TRUNC('month', b.booking_dt)::date AS fx_month,
    fx.rate_to_inr,
    CASE WHEN fx.rate_to_inr IS NULL THEN 1 ELSE 0 END AS is_fx_missing,
    b.gross_booking_value * fx.rate_to_inr AS gross_booking_value_inr,
    b.supplier_cost * fx.rate_to_inr AS supplier_cost_inr,
    b.discount_amount * fx.rate_to_inr AS discount_amount_inr,
    b.net_revenue * fx.rate_to_inr AS net_revenue_inr
FROM clean.fact_bookings AS b
LEFT JOIN analytics.fx_rates AS fx
    ON DATE_TRUNC('month', b.booking_dt)::date = fx.fx_month
   AND b.currency = fx.currency;
```

Why LEFT JOIN? Keep every booking even if an FX mapping is missing. The is_fx_missing flag makes the problem visible instead of silently dropping the booking.

Check	Verified result
fact_bookings_fx rows	119,877
Missing FX rows	0

Lesson 6: Validate booking conversion mathematically

WHAT TO DO
Compare stored INR values with local amount x rate_to_inr.

WHY
A successful SQL query does not prove the financial calculation is correct.

HOW
Use ABS() to detect any row where the difference exceeds a small tolerance.

```
SELECT COUNT(*) AS conversion_mismatch_rows
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0
  AND ABS(
        gross_booking_value_inr
        - (gross_booking_value * rate_to_inr)
      ) > 0.01;
```

```
SELECT COUNT(*) AS incorrect_inr_rows
FROM analytics.fact_bookings_fx
WHERE currency = 'INR'
  AND (
        rate_to_inr <> 1
        OR gross_booking_value_inr <> gross_booking_value
      );
```

Validation	Verified result	Interpretation
Conversion mismatch rows	0	Stored conversion matches the formula
Incorrect INR rows	0	INR always remains unchanged at rate 1

Lesson 7: Consolidated INR KPI formulas

WHAT TO DO	WHY	HOW
Calculate total bookings, total GBV, total net revenue, ABV and gross margin in INR.	FX normalization now makes company-wide aggregation valid.	Use SUM() on INR fields and reconcile every derived KPI mathematically.

```
SELECT
    COUNT(*) AS total_bookings,
    SUM(gross_booking_value_inr) AS total_gbv_inr,
    SUM(net_revenue_inr) AS total_net_revenue_inr,
    ROUND(SUM(gross_booking_value_inr) / COUNT(*), 2) AS avg_booking_value_inr,
    ROUND(
        (SUM(net_revenue_inr)
         / NULLIF(SUM(gross_booking_value_inr), 0)) * 100,
        2
    ) AS gross_margin_pct
FROM analytics.fact_bookings_fx
WHERE is_fx_missing = 0;
```

Important reconciliation checkpoint A previously entered KPI set did not reconcile: the reported ABV and gross-margin percentage did not match the reported total GBV, booking count and net revenue. Do not quote those totals as final until the exact KPI query is rerun and the formulas agree.

KPI	Correct formula
Average Booking Value (ABV)	Total GBV INR / Total Bookings
Gross Margin %	Total Net Revenue INR / Total GBV INR x 100

Analyst lesson Never trust a KPI merely because SQL returned a number. Related metrics must reconcile to each other mathematically.

5. Lesson 8: Normalize Cancellations and Refunds

8A: Check cancellation date coverage

WHAT TO DO	WHY	HOW
Compare cancellation dates with the FX table date range.	A cancellation can occur months after the original booking, including after the initial FX table ends.	Compare MIN/MAX cancellation_date with MIN/MAX fx_month before joining.

Check	Verified result
First cancellation date	2024-01-02
Last cancellation date	2026-05-11
Initial FX end month	2025-12-01

Finding The cancellation table extends five months beyond the original booking-based FX range, so the FX table must be extended through May 2026.

8B: Extend the FX table safely

```
INSERT INTO analytics.fx_rates (fx_month, currency, rate_to_inr)
SELECT
    m.fx_month::date,
    f.currency,
    f.base_rate_to_inr
FROM GENERATE_SERIES(
    (SELECT MAX(fx_month) + INTERVAL '1 month' FROM analytics.fx_rates),
    (SELECT DATE_TRUNC('month', MAX(cancellation_date))::date
     FROM clean.fact_cancellations),
    INTERVAL '1 month'
) AS m(fx_month)
CROSS JOIN analytics.fx_base_rates AS f;
```

Check	Verified result	Meaning
Total FX rows after extension	435	29 months x 15 currencies
Distinct months	29	Jan 2024 through May 2026
Distinct currencies	15	Coverage remains complete
Last FX month	2026-05-01	Covers latest cancellation month

8C: Convert refunds and cancellation fees

WHAT TO DO	WHY	HOW
Create analytics.fact_cancellations_fx.	Refunds and cancellation fees must use the booking currency but the cancellation month FX rate.	Join cancellation -> booking for currency, then month + currency -> fx_rates.

```
CREATE TABLE analytics.fact_cancellations_fx AS
SELECT
    c.*,
    DATE_TRUNC('month', c.cancellation_date)::date AS fx_month,
    b.currency,
    fx.rate_to_inr,
    CASE WHEN fx.rate_to_inr IS NULL THEN 1 ELSE 0 END AS is_fx_missing,
    c.refund_amount * fx.rate_to_inr AS refund_amount_inr,
    c.cancellation_fee * fx.rate_to_inr AS cancellation_fee_inr
FROM clean.fact_cancellations AS c
LEFT JOIN clean.bookings_deduped AS b
    ON c.booking_id = b.booking_id
LEFT JOIN analytics.fx_rates AS fx
    ON DATE_TRUNC('month', c.cancellation_date)::date = fx.fx_month
   AND b.currency = fx.currency;
```

Check	Verified result
Cancellation FX rows	13,972
Missing FX rows	0

Event-date rule Refunds are converted with cancellation-month FX, not booking-month FX, because the refund is a cancellation-time financial event.

6. Lesson 9: Normalize Payments at Transaction Level

9A: Check payment date coverage

Check	Verified result
First payment date	2024-01-01
Last payment date	2026-01-04
FX coverage end	2026-05-01

The payment range is fully inside the extended FX range, so no additional FX months are required.

9B: Convert each payment transaction before aggregation

WHAT TO DO	WHY	HOW
Create analytics.payment_transactions_fx with an INR amount for every payment transaction.	A single booking may have multiple payments in different months. Aggregating local currency first and applying one rate can be wrong.	Join each transaction to the booking currency and the payment-month FX rate, then calculate amount_inr.

```
CREATE TABLE analytics.payment_transactions_fx AS
SELECT
    p.*,
    b.currency,
    DATE_TRUNC('month', p.payment_dt)::date AS fx_month,
    fx.rate_to_inr,
    CASE WHEN fx.rate_to_inr IS NULL THEN 1 ELSE 0 END AS is_fx_missing,
    p.amount_num * fx.rate_to_inr AS amount_inr
FROM staging.payments_typed AS p
LEFT JOIN clean.bookings_deduped AS b
    ON p.booking_id = b.booking_id
LEFT JOIN analytics.fx_rates AS fx
    ON DATE_TRUNC('month', p.payment_dt)::date = fx.fx_month
   AND b.currency = fx.currency;
```

Validation	Verified result
Total payment transaction rows	127,189
Missing FX rows	0
Missing currency rows	0
Conversion mismatch rows	0

Core payment rule Convert transaction first -> aggregate later. This preserves the correct FX rate for each payment date.

9C: Aggregate converted payments to booking grain

```
CREATE TABLE analytics.payment_booking_summary_fx AS
SELECT
    booking_id,
    COUNT(*) AS payment_rows,
    SUM(amount_inr) AS total_payment_amount_inr,
    SUM(CASE WHEN payment_status_std = 'Paid' THEN amount_inr ELSE 0 END)
        AS paid_amount_inr,
    SUM(CASE WHEN payment_status_std = 'Refunded' THEN amount_inr ELSE 0 END)
        AS refunded_amount_inr,
    MIN(payment_dt) AS first_payment_date,
    MAX(payment_dt) AS last_payment_date
FROM analytics.payment_transactions_fx
WHERE is_fx_missing = 0
GROUP BY booking_id;
```

Check	Verified result	Meaning
Summary rows	120,000	One row for every real booking
Unique booking IDs	120,000	Booking grain confirmed
Duplicate booking IDs	0	Safe to join to booking-level analysis
B0088132 payment_rows	2	Known multi-payment example preserved correctly

7. Lesson 10: Reconcile Payments with Booking Validity

WHAT TO DO
Compare payment summaries to both all real bookings and the final analytical fact table.

WHY
Existence and analytical validity are different concepts. A payment can belong to a real booking that was excluded from KPI analysis.

HOW
Use clean.bookings_deduped to test true existence and clean.fact_bookings to test analytical eligibility.

```
SELECT
    COUNT(*) AS total_payment_bookings,
    COUNT(*) FILTER (WHERE b.booking_id IS NOT NULL)
        AS payments_linked_to_valid_bookings,
    COUNT(*) FILTER (WHERE b.booking_id IS NULL)
        AS payments_not_in_clean_fact
FROM analytics.payment_booking_summary_fx AS p
LEFT JOIN clean.fact_bookings AS b
    ON p.booking_id = b.booking_id;
```

Check	Verified result	Interpretation
Total payment bookings	120,000	All booking-level payment summaries
Linked to valid clean fact	119,877	Eligible for normal booking KPI analysis
Not in clean fact	123	Real bookings excluded for serious data-quality issues
True orphan payment bookings	0	No payment summary points to a nonexistent booking

Concept 123 payment summaries are not orphans. They belong to the 123 real bookings excluded from clean.fact_bookings because of serious business-rule problems.

8. Lesson 11: Final FX Validation Framework

WHAT TO DO	WHY	HOW
Validate FX coverage, INR identity rates and record grain across every normalized table.	A bad FX join can silently create missing mappings or duplicate rows even when the multiplication formula is correct.	Run missing-FX counts, INR rate checks and row-vs-distinct-key checks.

```
SELECT
    (SELECT COUNT(*) FROM analytics.fact_bookings_fx
     WHERE is_fx_missing = 1) AS booking_missing_fx,
    (SELECT COUNT(*) FROM analytics.fact_cancellations_fx
     WHERE is_fx_missing = 1) AS cancellation_missing_fx,
    (SELECT COUNT(*) FROM analytics.payment_transactions_fx
     WHERE is_fx_missing = 1) AS payment_missing_fx;
```

```
SELECT COUNT(*) AS incorrect_inr_fx_rows
FROM analytics.fx_rates
WHERE currency = 'INR'
  AND rate_to_inr <> 1;
```

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT booking_id) AS unique_booking_ids,
    COUNT(*) - COUNT(DISTINCT booking_id) AS duplicate_rows
FROM analytics.fact_bookings_fx;
```

Expected logic: all missing-FX counts should be 0; INR rates should always equal 1; and the FX join must preserve the original fact-table grain.

Validation note The conversation moved forward before these final combined checks were reported. They should be rerun and recorded before the project is published.

9. Lesson 12: Final KPI Reconciliation

The final Phase 3 KPI query should be rerun after all FX tables are complete. The goal is not just to produce totals, but to verify that derived metrics equal their underlying formulas.

Metric	Reconciliation formula	Required check
Total Bookings	COUNT(*)	Should equal 119,877 in fact_bookings_fx
Total GBV INR	SUM(gross_booking_value_inr)	Use only rows with valid FX
Total Net Revenue INR	SUM(net_revenue_inr)	Use the same filtered population as GBV
ABV INR	Total GBV INR / Total Bookings	Must exactly match query output after rounding
Gross Margin %	Total Net Revenue INR / Total GBV INR x 100	Must exactly match query output after rounding

Do not publish unreconciled KPI totals A prior manually entered set had an order-of-magnitude mismatch in ABV and margin. The formulas above are the source of truth; rerun the SQL and use only reconciled values.

10. Phase 3 Table Inventory and Grain

Table	Grain	Purpose
analytics.fx_base_rates	1 row per currency	Base synthetic INR conversion rate
analytics.fx_rates	1 row per month + currency	Time-aware FX lookup
analytics.fact_bookings_fx	1 row per valid booking	Booking financials in local currency and INR
analytics.fact_cancellations_fx	1 row per clean cancellation	Refund and cancellation fee in INR
analytics.payment_transactions_fx	1 row per payment transaction	Transaction-level payment amount in INR
analytics.payment_booking_summary_fx	1 row per booking	Safe booking-grain payment summary in INR

Grain rule Before every join, say out loud what one row represents in each table. Most double-counting errors begin when two different grains are joined without aggregation.

11. SQL Concepts Learned in Phase 3

Concept	Beginner meaning
DATE_TRUNC()	Convert a transaction date to its month key for monthly FX joins.
GENERATE_SERIES()	Generate a continuous list of monthly dates.
CROSS JOIN	Create every month x currency combination.
LEFT JOIN	Preserve source transactions while surfacing missing FX or currency mappings.
CASE	Create missing-FX flags and conditional payment totals.
SUM()	Aggregate normalized financial amounts.
MIN() / MAX()	Check date coverage and first/last transaction dates.
COUNT(DISTINCT ...)	Validate grain and uniqueness.
ABS()	Measure conversion mismatch regardless of sign.
NULLIF()	Protect financial ratios from division by zero.
GROUP BY	Move from transaction grain to booking grain.

12. Analyst Thinking: Why These Decisions Matter

Do not sum mixed currencies

A total is only meaningful when every amount has the same unit.

Preserve local and reporting values

Keeping both values allows auditing, debugging and explanation of FX effects.

Use event-specific dates

Booking, cancellation and payment values can happen in different months and therefore use different rates.

Convert before aggregating payments

Multiple payment dates can map to different FX months. Aggregating local amounts first loses that timing information.

Extend lookup coverage before joining

A missing lookup month should be fixed explicitly, not hidden by dropped rows.

Flag missing FX instead of dropping rows

A LEFT JOIN plus is_fx_missing makes data loss visible.

Reconcile derived KPIs

ABV and margin should be recomputable from the totals shown next to them.

13. Common Mistakes to Avoid

- Summing GBV across currencies before normalization.
- Using a single lifetime FX rate when the project is designed for monthly rates.
- Joining FX only on currency and forgetting the month key.
- Using booking-date FX for refunds or payment transactions.
- Aggregating payment amounts first and converting the total with one FX rate.
- Using INNER JOIN for FX conversion and silently losing transactions with missing rates.
- Failing to extend the FX table when later cancellations go beyond the booking date range.
- Averaging row-level margin percentages to calculate the company-wide margin.
- Treating payments linked to excluded analytical bookings as true orphan payments.
- Publishing KPI numbers that do not reconcile to the formulas.

14. Reusable Phase 3 Checklist

- Choose and document the reporting currency.
- List every source currency and confirm coverage.
- Determine the full date range for every financial event table.
- Create a controlled FX lookup at the required time grain.
- Validate month x currency uniqueness in the FX table.
- Convert booking values using booking-month FX.
- Validate conversion formula and INR identity behavior.
- Extend FX coverage for later cancellations when required.
- Convert refund and cancellation amounts using cancellation-month FX.
- Convert payment transactions using payment-month FX before aggregation.
- Aggregate converted payment transactions to booking grain.
- Reconcile payments to all real bookings and to valid analytical bookings separately.
- Check missing FX, duplicate grains and INR rate = 1.
- Reconcile ABV and gross margin to their underlying totals before publishing.

15. Quick SQL Reference

Create a month key

```
DATE_TRUNC('month', transaction_date)::date
```

Generate monthly dates

```
GENERATE_SERIES(start_month, end_month, INTERVAL '1 month')
```

Convert local amount to INR

```
local_amount * rate_to_inr AS amount_inr
```

Flag missing FX

```
CASE WHEN fx.rate_to_inr IS NULL THEN 1 ELSE 0 END AS is_fx_missing
```

Validate conversion

```
ABS(amount_inr - (local_amount * rate_to_inr)) > 0.01
```

Validate one row per business key

```
COUNT(*) - COUNT(DISTINCT business_key) AS duplicate_rows
```

Safe ratio

```
(numerator / NULLIF(denominator, 0)) * 100
```

16. Interview Questions You Can Now Answer

Question	Strong answer
Why can you not sum revenue across multiple currencies directly?	Because the values have different units. They must first be converted into a common reporting currency using an appropriate FX rate.
What is the grain of your FX table?	One row per month per currency.
Why did you use a CROSS JOIN?	To pair every required month with every supported currency.
Why did you use LEFT JOIN for FX conversion?	To preserve all source transactions and explicitly flag missing FX mappings instead of silently dropping data.
Why do bookings, cancellations and payments use different FX dates?	Each monetary value belongs to a different economic event, so the conversion date should match that event.
Why convert payments before aggregating?	Multiple payment transactions for one booking can occur in different months and therefore require different FX rates.
How did you validate the FX conversion?	I checked missing mappings, conversion formula mismatches, INR identity rates and preservation of fact-table grain.
What is the difference between an orphan payment and a payment for an excluded booking?	An orphan points to no real booking. A payment for an excluded booking still points to a real booking that failed analytical quality rules.

17. Phase 3 Completion Status and Phase 4 Handoff

Area	Status	Evidence
Analytics schema	Complete	analytics objects created
Base currency mapping	Complete	15 currencies
Monthly FX coverage	Complete	435 rows, 29 months, through May 2026
Booking FX conversion	Complete	119,877 rows, 0 missing FX, 0 conversion mismatches
Cancellation FX conversion	Complete	13,972 rows, 0 missing FX
Payment transaction FX conversion	Complete	127,189 rows, 0 missing FX/currency, 0 mismatches
Booking-level payment summary	Complete	120,000 unique booking IDs, 0 duplicates
Payment reconciliation	Complete	119,877 valid + 123 excluded = 120,000; 0 true orphans
Final consolidated KPI reconciliation	Re-run before publishing	Prior manually entered KPI set did not reconcile

Phase 4 handoff The FX-normalized tables are ready to support business data marts such as monthly performance, agent performance, destination performance, supplier performance, hotel performance and cancellation analysis.

Recommended first Phase 4 mart

analytics.monthly_performance - one row per booking month with total bookings, completed bookings, cancelled bookings, GBV INR, net revenue INR, ABV, gross margin percentage and cancellation rate.

18. Phase 3 One-Page Mental Model

RAW currencies -> CLEAN trusted transactions -> FX month + currency lookup -> INR-normalized facts -> validated booking-grain summaries -> Phase 4 data marts

If the money is...	Use this date	Then do this
Booking GBV / supplier cost / discount / net revenue	booking_dt	Convert each booking row to INR
Refund / cancellation fee	cancellation_date	Convert each cancellation row to INR
Payment amount	payment_dt	Convert each transaction to INR, then aggregate by booking

Final principle Financial normalization is not just multiplication. It is a controlled data-modeling process: choose the right grain, choose the right event date, preserve traceability, validate coverage, prevent row multiplication and reconcile the final KPIs.